

Lab Assignment

C17 — Springfield Power Plant Employee Database

Topics: STL Vector • File I/O • Static Members • Template Functions • Exception Handling • Business Rules

1. Objective

This lab builds a memory-resident employee database for the Springfield Nuclear Power Plant. You will read structured records from a CSV file into a `vector<Employee>`, enforce business rules via exception handling, and query the database using template functions and STL algorithms. The lab integrates five major C++ topics studied so far into a single realistic application.

By the end of this lab you will be able to:

- ▶ Design a class with static data members, static constants, and a factory method.
- ▶ Enforce business rules by throwing exceptions from setter methods.
- ▶ Read a comma-separated file and populate an STL vector safely.
- ▶ Write template functions that operate on a vector of any type.
- ▶ Query, filter, sort, and aggregate a vector using range-for loops and STL algorithms.

2. Provided Files

File	Description
Employee.h	Complete Employee class — study this before writing any experiment.
main.cpp	Starter file with <code>makeDatabase()</code> and <code>showVector()</code> already implemented.
PowerPlantEmployees.txt	CSV data file — 100 employee records, one per line.

2.1 CSV Record Format

Each line in `PowerPlantEmployees.txt` follows this structure:

```
ID,Name,Age,Salary,Department
103,Homer Simpson,45,52000.00,Sector 7-G
120,Professor Frink,58,88000.00,Research
101,Montgomery Burns,104,1000000.00,Executive
```

2.2 Employee Class — Key Members

The provided `Employee.h` includes the following. Study each one before writing your experiments:

Member	Description
<code>static int deathCounter</code>	Tracks number of calls to the Employee's destructor. (To be used to reduce trashing)
<code>static const int MAX_AGE = 75</code>	Upper bound for valid employee age. Burns is exempt by name check.
<code>static const int MIN_AGE = 18</code>	Lower bound for valid employee age.
<code>static const int MAX_NAME_LEN = 20</code>	Names longer than this are rejected by <code>setName()</code> .
<code>setAge(int)</code>	Throws <code>runtime_error</code> if age is out of range. Montgomery Burns is exempt.
<code>setSalary(double)</code>	Throws <code>runtime_error</code> if salary exceeds \$999,999 and employee is not Burns.
<code>makeEmployeeFromString(...)</code>	Static factory method. Parses five string arguments and returns an Employee.
<code>toString()</code>	Returns a formatted string representation. Virtual for future inheritance.
<code>operator<<</code>	Delegates to <code>toString()</code> . Works with <code>cout</code> directly.

3. Required Experiments

Experiment 01 — Business Rules and Exception Handling

Test the Employee constructor directly without using the CSV file. Create Employee objects that trigger each of the three business rules and confirm the correct exception is thrown and caught.

Tasks to verify:

- ▶ Create a valid Employee — confirm it prints correctly.
- ▶ Create an Employee whose name exceeds `MAX_NAME_LEN` characters — confirm the rejection message.
- ▶ Create an Employee whose age is below `MIN_AGE` — confirm the rejection message.
- ▶ Create an Employee whose age is above `MAX_AGE` — confirm the rejection message.
- ▶ Create Montgomery Burns with age 104 and salary \$1,000,000 — confirm he is accepted despite both values exceeding the normal limits.
- ▶ Verify that `Employee::counter` reflects the correct number of live objects after construction and destruction.

```

void experiment01() {
    try {
        // Valid employee
        Employee e1("103", "Homer Simpson", 45, 52000.00, "Sector 7-G");
        cout << "e1: " << e1 << endl;

        // Name too long (exceeds MAX_NAME_LEN)
        Employee e2("999", "Bartholomew JoJo Simpson Jr.", 10, 1000.00, "HR");
        cout << "e2: " << e2 << endl;
    }
    catch (const exception& e) {
        cout << "Caught: " << e.what() << endl;
    }

    // Montgomery Burns - exempt from all rules
    try {
        Employee burns("101", "Montgomery Burns", 104, 1000000.00, "Executive");
        cout << "Burns: " << burns << endl;
    }
    catch (const exception& e) {
        cout << "Burns rejected (unexpected): " << e.what() << endl;
    }

    cout << "Live employees: " << Employee::counter << endl;
}

```

Experiment 02 — Load the Database from File

Call the provided `makeDatabase()` function to populate the vector from `PowerPlantEmployees.txt`. Then print the full database using the provided `showVector()` template function. Observe how many records were accepted and how many were rejected by business rules.

Tasks to verify:

- ▶ Call `makeDatabase(db)` and confirm the total number of records loaded.
- ▶ Call `showVector(db, "Power Plant Employees")` to print all accepted employees.
- ▶ Count and explain every rejection message printed during loading.
- ▶ Print `Employee::counter` after loading and explain the value.

Experiment 03 — Filter by Department

Scan the database and extract employees belonging to a specific department into a new vector. Call `showVector()` on the filtered result.

Tasks:

- ▶ Filter employees in the "Safety" department into `vSafety`.
- ▶ Filter employees in the "Research" department into `vResearch`.
- ▶ Filter employees older than 50 into `vSenior`.
- ▶ Print each filtered vector using `showVector()`.

```

void experiment03(vector<Employee>& db) {
    vector<Employee> vSafety;
    for (const auto& emp : db) {
        if (emp.getDepartment() == "Safety")
            vSafety.push_back(emp);
    }
    showVector(vSafety, "Safety Department");

    // TODO: repeat for Research and Senior (age > 50)
}

```

Experiment 04 — Salary Statistics

Write a template function `vectorStats()` that computes the minimum, maximum, and average of a numeric field extracted from a vector. Apply it to compute salary statistics for the full database and for individual departments.

Function signature:

```

// Computes and prints min, max, and average of values in a vector
template <typename T>
void vectorStats(const vector<T>& v, const string& caption);

```

Tasks:

- ▶ Build a `vector<double>` of all salaries from the full database.
- ▶ Call `vectorStats()` on it — print min, max, and average salary.
- ▶ Repeat for only the Engineering department salaries.
- ▶ Repeat for only the Executive department salaries.

Experiment 05 — Sort the Database

Use `std::sort` with custom lambda comparators to sort the employee database in different ways. Print the result after each sort using `showVector()`.

Sort tasks:

- ▶ Sort by **salary descending** — highest paid first.
- ▶ Sort by **age ascending** — youngest first.
- ▶ Sort by **name alphabetically** — A to Z.
- ▶ Sort by **department then name** — group by department, alphabetical within each group.

```

#include <algorithm>

// Sort by salary descending
sort(db.begin(), db.end(), [](const Employee& a, const Employee& b) {
    return a.getSalary() > b.getSalary();
});
showVector(db, "Sorted by Salary (Highest First)");

// TODO: add sorts by age, name, and department+name

```

Experiment 06 — Search the Database

Write a template function `findByField()` that searches the database for employees matching a given criterion. Use it to find specific employees by name, department, and salary range.

Search tasks:

- ▶ Find and print the employee named "Homer Simpson".
- ▶ Find all employees earning more than \$80,000.
- ▶ Find the employee with the highest salary in the database.
- ▶ Count how many employees work in the "Maintenance" department.
- ▶ Use `std::find_if` to locate the first employee over age 70.

4. Implementation Notes

- ▶ **Pass vectors by const reference:** `showVector()` and all query functions should take `const vector<Employee>&` — not by value. Passing by value copies the entire database and triggers destructors on return.
- ▶ **Use const auto& in range-for loops:** `for (const auto& emp : db)` avoids copying each Employee object during iteration.
- ▶ **Constructor setter order matters:** `setDepartment()` must be called before `setSalary()` because the salary validator checks the department name.
- ▶ **The counter:** `Employee::counter` increments on construction and decrements on destruction. After loading 200 records and rejecting some, the counter reflects only the currently live Employee objects in all vectors combined.
- ▶ **Burns exemption:** Montgomery Burns is exempt from age and salary limits by name check inside the setters. His record in the CSV (age 104, salary \$1,000,000) will load successfully.

Common mistakes to avoid

- ▶ Passing `vector<Employee>` by value to `showVector()` — causes unnecessary copies and inflates the destructor counter.
- ▶ Calling `setSalary()` before `setDepartment()` in the constructor — the salary business rule always sees an empty department string.
- ▶ Using `auto emp` instead of `const auto& emp` in filter loops — copies every element unnecessarily.
- ▶ Not wrapping each record parse in its own try-catch inside `makeDatabase()` — one bad record would abort loading the entire file.

5. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. Why is the factory method `makeEmployeeFromString()` declared static? What would happen if it were a regular member function?
- ▶ 2. Why does `Employee::counter` not equal the number of lines in the CSV file after loading?
- ▶ 3. Explain why the setter order in the constructor matters for the salary business rule.
- ▶ 4. What is the advantage of wrapping each record parse inside its own try-catch block inside the loading loop, rather than wrapping the entire loop?
- ▶ 5. Why does passing a vector by value to a function that only reads it cause unnecessary destructor calls?

Key design insight

This application demonstrates how the major C++ topics combine in a real program: exceptions enforce business rules at construction time, static members track global state, the factory method cleanly separates parsing from object creation, template functions let one implementation work across many data types, and STL algorithms replace manual loops with readable, tested code. Each piece was studied in isolation — this lab shows how they work together.